

MySupplier

Developer Handover: what is built, how to run it, what remains

Build for less. Saudi construction materials: price discovery, RFQ and bidding, BOQ research, e-commerce, multi-supplier portal, payments, delivery and e-invoicing.

Repository	github.com/invetechs1/My-supplier-
Branch	claude/building-materials-ecommerce-ktfxj3
Document date	25 September 2026
Audience	The developer taking the platform to production
Companion documents	deploy/GO-LIVE.md (runbook), docs/API.md (REST contract), docs/ARCHITECTURE.md, docs/ROADMAP.md, README.md

1. Executive summary

The platform is code-complete for launch. Every feature described below is implemented, unit-tested, exercised end to end (API smoke script plus browser smoke test) and wired into CI. Nothing that remains is a missing feature: the remaining work is **external accounts and credentials** (payment gateway, SMS, carriers, ZATCA, app stores), **real business data** (suppliers, legal entity details) and a short list of **hardening items** that only matter at scale.

Read this document top to bottom once, then work through section 6 in the order given. A realistic plan for one developer is about three weeks to a public launch, with most calendar time spent waiting on third parties (Moyasar approval, Apple review, ZATCA onboarding).

Status at a glance

Area	Status	Notes
Public catalogue, price comparison, price history	DONE	270 seeded materials incl. equipment and machinery, 23 demo suppliers, 32 categories
RFQ and bidding (buyer posts, suppliers bid, award creates order)	DONE	Notifications by in-app, push and email
BOQ research (paste a bill of quantities, get prices per supplier)	DONE	Heuristic matcher plus optional AI extraction
E-commerce storefront, cart, checkout, orders	DONE	Multi-supplier cart, VAT 15 percent, delivery fees, COD, bank transfer, card
Supplier portal (dashboard, orders, inventory, team, branches, finance)	DONE	Roles OWNER, MANAGER, SALES, WAREHOUSE; commission and payouts
AI price collection (PDF, Excel, photo, quotations, web feeds, outreach links)	CREDENTIALIALS	Code done; needs a rotated Anthropic API key in production
Admin console (full e-commerce back office)	DONE	Reports and analytics, orders, payments ledger and refunds, coupons and promotions, review moderation, support inbox, catalogue, imports, outreach, feeds, users, companies and verification, payouts, shipping rates, announcements, audit log, settings
Payments (Moyasar: Mada, Visa, Mastercard, Apple Pay), refunds, webhooks	CREDENTIALIALS	Code done; needs a Moyasar merchant account and keys
Phone OTP login (Unifonic SMS)	CREDENTIALIALS	Code done; falls back to console logging without keys
Delivery: zones, rate cards, quotes, shipments, tracking, carrier webhooks	DEV WORK	Rate-card quoting works today; live carrier booking needs contract-specific payload mapping
E-invoicing (ZATCA phase 1 QR done; phase 2 UBL XML and reporting)	DEV WORK	XAdES signing and Fatoora onboarding remain
Web app (Next.js, 45+ routes, Arabic and English content, SEO, legal pages)	DONE	Legal pages contain placeholders for the real company details
Mobile app (Expo, iOS and Android, same features as web)	CREDENTIALIALS	Code done; needs Apple and Google developer accounts and EAS builds
Deployment (Docker Compose, Caddy HTTPS, backups, CI)	DONE	Needs a server, domain and the deploy/.env file
Product discovery: spec facets, fuzzy search, autocomplete, brands, reviews, Q&A, wishlists, price alerts	DONE	Category attribute definitions drive filters and spec tables

Area	Status	Notes
B2B commerce: quantity tiers, sale prices, address book, credit terms, buy again, returns (RMA), recurring orders	DONE	Credit lines are approved per company by the admin
ERP integration: scoped API keys, /integrations/v1 sync API, signed webhooks, OpenAPI spec, integration guide	DONE	Suppliers and buyers manage keys and webhooks in their consoles
Security audit fixes	DONE	See section 8 for the small optional backlog

2. Architecture and repository layout

A pnpm monorepo. One PostgreSQL database, one REST API, one web app and one mobile app that both consume the API. Shared TypeScript types keep the three in sync.

Path	What it is	Stack
apps/api	REST API, background jobs (outreach, feeds), file uploads, AI extraction, payments, shipping, e-invoicing	Node 20, Express 4, TypeScript, Prisma 5, PostgreSQL 16, zod, JWT, Anthropic SDK, Sentry
apps/web	Public site, buyer dashboard, supplier portal, admin console	Next.js 14 app router, Tailwind, pure SVG charts, Playwright e2e
apps/mobile	iOS and Android app (and a web export used for previews)	Expo SDK 51, expo-router, secure-store, notifications, image and document pickers
packages/shared	Domain types shared by API, web and mobile	TypeScript
docs/	API contract, architecture, roadmap, this handover	Markdown
deploy/	Production compose file, Caddyfile, env template, deploy, backup, restore and demo-seed scripts, go-live runbook	Docker Compose, Caddy
.github/workflows/ci.yml	CI: api (tests, seed, smoke), web build, mobile typecheck, docker build, browser e2e	GitHub Actions

Domain model (Prisma)

Company (multi-tenant, verified flag, rating, commission), User (roles ADMIN, BUYER, SUPPLIER plus company role), Category, Material, PriceListing (sources SUPPLIER, MARKET, IMPORTED, QUOTATION) and PriceHistory, RFQ, Bid, BidItem, Order, OrderItem, OrderEvent, OrderMessage, Cart, Payment, Payout, Review, Notification, Device (push tokens), OtpCode, PasswordResetToken, CompanyInvite, CompanyDocument, Branch, StockMovement, PriceImport and PriceImportRow (AI review queue), Feed, PriceUpdateRequest (magic links), Shipment, ShipmentEvent, ShippingRate, EInvoiceRecord, PlatformSetting.

Schema: apps/api/prisma/schema.prisma. Migrations are committed under apps/api/prisma/migrations and applied with prisma migrate deploy.

Key backend modules

File (apps/api/src/...)	Responsibility
app.ts	Express wiring: helmet, CORS, rate limits, optional auth, all routers, static uploads, Sentry and error handlers

File (apps/api/src/...)	Responsibility
middleware/auth.ts	Bearer JWT; query token accepted only on download routes; requireAuth, requireCompany, signToken
lib/serialize.ts, lib/security.ts, lib/uploads.ts	Private field stripping, SSRF guard, CSV neutralisation, phone normalisation, magic-byte upload validation
services/pricing.ts, catalog.ts, shop.ts	Price aggregation, best price, history, product and offer enrichment
services/boq.ts	BOQ text parsing, tokenising with synonyms and sizes, matching, cheapest-basket optimisation
services/ai.ts, imports.ts, catalogImport.ts	Claude extraction with JSON schema, heuristic fallback, review queue, publishing rows into listings, feed runner
services/portal.ts	Company roles, settings, commission, stock movements, order events, ratings
services/payments.ts, routes/payments.ts	Moyasar intent, verify, webhook, refunds, hosted pay page
services/shipping.ts, services/carriers/index.ts	Zones, rate cards, quotes, shipment lifecycle, carrier adapters and webhooks
services/einvoice.ts, services/zatca.ts	UBL 2.1 XML, hash chain, phase 1 TLV QR, phase 2 reporting call
services/notifications.ts, mailer.ts, push.ts, sms.ts	In-app, email (SMTP), Expo push, Unifonic SMS

3. Running the project locally

- 1 Install Node 20 or newer, pnpm 10 (corepack enable) and PostgreSQL 16 (Docker is fine).
- 2 Clone the repository and check out the branch named on the cover page. Run `pnpm install` at the root.
- 3 Copy `apps/api/.env.example` to `apps/api/.env` and set `DATABASE_URL` and `JWT_SECRET`. Optional keys (Anthropic, SMTP, Moyasar) can stay empty; the code degrades gracefully.
- 4 Apply migrations and seed demo data, then start the API, the web app and, optionally, the mobile app.

```
pnpm install
cd apps/api && pnpm prisma migrate deploy && pnpm prisma generate && pnpm prisma:seed
pnpm dev:api      # http://localhost:4000 (health: GET /api/v1/health)
pnpm dev:web      # http://localhost:3000
pnpm dev:mobile   # Expo dev server; press i / a / w
```

Demo accounts created by the seed

Email	Password	Role
admin@mysupplier.sa	Admin123!	Platform admin
buyer@mysupplier.sa	Buyer123!	Buyer (contractor company)
supplier@mysupplier.sa	Supplier123!	Supplier company OWNER
warehouse@mysupplier.sa	Supplier123!	Supplier company WAREHOUSE role
sales@<supplier-slug>.sa	Supplier123!	OWNER of each of the other 17 demo suppliers

Delete or change these before the public launch (see task 6.14).

Tests and checks a contributor runs before pushing

```
pnpm -r test # 36 unit tests in apps/api/tests
cd apps/api && bash scripts/smoke.sh # API end-to-end: RFQ -> bid -> award -> order, BOQ, shop checkout, AI import
cd apps/web && pnpm build && pnpm e2e # Next build and Playwright browser smoke (WEB_URL, API_URL env)
```

```
cd apps/mobile && pnpm typecheck
```

CI runs the same jobs on every push (.github/workflows/ci.yml). Keep it green.

Database changes

Edit schema.prisma, then generate a migration file with `prisma migrate dev --name <name>` (interactive) or, in non-interactive environments, `prisma migrate diff --from-url $DATABASE_URL --to-schema-datamodel prisma/schema.prisma --script` written to a new folder under prisma/migrations, followed by `migrate deploy` and `generate`.

4. Deploying to production

Production is a single VM running Docker Compose: Caddy (automatic HTTPS), api, web, PostgreSQL and a nightly backup container. The full runbook is **deploy/GO-LIVE.md**; this is the short version.

- 1 Provision an Ubuntu 22.04 or 24.04 VM in Saudi Arabia (Riyadh region on AWS, Azure, Oracle or STC Cloud), 2 vCPU and 4 GB RAM minimum, with Docker installed and ports 80 and 443 open.
- 2 Point DNS A records for the site domain and the api sub-domain at the VM.
- 3 Clone the repository on the server, copy `deploy/.env.production.example` to `deploy/.env` and fill it in (see the variable table in section 7).
- 4 Run `deploy/scripts/deploy.sh`. It builds the images, starts the stack, applies migrations and prints the health check.
- 5 Optionally load demo data with `deploy/scripts/seed-demo.sh` for staging only. Do not seed production.
- 6 Verify: `https://api.DOMAIN/api/v1/health` returns ok, the home page loads, login works, a test order can be placed with cash on delivery.
- 7 Schedule `deploy/scripts/backup.sh` (the compose file already runs a nightly dump) and test `restore.sh` once on staging.

```
git clone <repo> /opt/mysupplier && cd /opt/mysupplier
cp deploy/.env.production.example deploy/.env && nano deploy/.env
bash deploy/scripts/deploy.sh
docker compose -f deploy/docker-compose.prod.yml logs -f api
```

Redeploying after a code change is the same `deploy.sh` command. Migrations run automatically on start.

5. What is built, module by module

Public site and buyer features

- **Catalogue and price comparison:** browse by category, search, material page with all supplier offers, best price, price history chart, supplier profiles with ratings and reviews.
- **BOQ research:** paste or upload a bill of quantities; each line is matched to a material, priced across suppliers and grouped into the cheapest basket per supplier with a where-to-buy summary. Arabic and English units and synonyms are handled.
- **RFQ and bidding:** buyers post an RFQ with items, delivery city and deadline; matching suppliers are notified, bid per line, and the buyer awards. Awarding creates an order.
- **Shop:** Amazon-style storefront (construction materials only), product pages with offers from all suppliers, cart split by supplier, delivery quotes per supplier, checkout with cash on delivery, bank transfer or card, buyer quotations, order tracking, messaging with the supplier, reviews, invoices (with ZATCA QR) and refunds.
- **Accounts:** email and password, phone OTP login, forgot and reset password, company invites, notification centre, push notifications, account deletion.

- **Discovery:** filters built from per-category attribute definitions (sizes, grades, power, capacity, resolution and so on) with counts, brand and price facets, fuzzy search with suggestions, brands directory, product reviews with verified-purchase badges and supplier replies, questions and answers, frequently bought together, recommendations and recently viewed, saved lists and price or back-in-stock alerts.
- **B2B buying:** quantity price tiers and time-limited sale prices, saved site addresses, purchase-order numbers, credit terms (net days against an approved limit), buy again and reorder, returns with approval, receipt and refund, recurring orders for consumables.
- **Content:** about, help, contact, terms, privacy, refund policy, sitemap, robots, PWA manifest.

Supplier portal (multi-tenant)

- **My products:** every offer the supplier has in the marketplace as buyers see it, with photo, price, stock, minimum quantity and lead time editable in place, pause and resume, competitor price and cheapest-in-city flag, units sold in 30 days, and a link to the public product page. Available on web and mobile.
- Dashboard with revenue, orders, response time, rating and stock alerts; order list with status changes, events timeline, delivery notes, shipments and messages.
- Inventory: per-material stock, stock movements, low-stock alerts; price list editing; bulk price imports by PDF, Excel, photo or web page with an AI-extracted review queue before publishing.
- Company profile and verification documents (CR, VAT), branches, team members with roles, finance page with commission due, payout history and statement export.

Admin console

- **Overview and reports:** KPIs, GMV and order trends with period-over-period change, average order value, RFQ conversion, top products, suppliers, categories and cities, payment-method split, CSV export.
- **Commerce:** order oversight, payments ledger with refunds and outstanding balances, coupons and promotions (percent or fixed, minimum order, cap, validity window, usage limit; applied at checkout and split across supplier orders), review moderation (hide, reply, delete), support inbox fed by the public contact form.
- **Catalogue and pricing:** materials and categories, AI import review across suppliers, outreach (magic price-update links to suppliers with stale prices), feeds (JSON, CSV or HTML pages polled on a schedule).
- **Users and partners:** users and roles, companies and document verification, payouts, shipping rate cards.
- **System:** announcements to all buyers or suppliers (in-app, push, optional email), append-only audit log of every privileged action, platform settings (commission, VAT, delivery).

Integrations already coded

- **ERP integration:** scoped API keys per company, a versioned sync API (catalogue, bulk price and stock push, order pull and status update, purchases, e-invoices, RFQs and bids), outbound webhooks signed with HMAC and retried with backoff, an OpenAPI 3 specification (docs/openapi.yaml) and an integration guide with SAP, Oracle, Dynamics, Odoo and Zoho field mappings (docs/INTEGRATIONS.md).
- **Moyasar** payments: intent creation bound to the order, client verification, signed webhooks, refunds, hosted pay page for mobile.
- **Unifonic** SMS for OTP; **SMTP** email with HTML templates; **Expo** push; **Sentry** error reporting on API, web and mobile; **Anthropic Claude** for document extraction.
- **Carriers:** zone model (Riyadh, Jeddah, Dammam and Eastern, other), rate cards by weight and pallet, quotes at checkout, shipment records with tracking timeline, generic REST adapter skeletons for TruKker, Trella, SMSA, Aramex and SPL, inbound status webhook.

- **ZATCA:** phase 1 TLV QR on every invoice; phase 2 UBL 2.1 XML with hash chain and a reporting call that activates once credentials are set.

Security measures in place

- Helmet, CORS allow-list, rate limits on auth, OTP and BOQ; bcrypt passwords; JWT with expiry; private fields (IBAN, commission, verification notes, token hashes) never serialised to clients.
- Uploads validated by magic bytes and size, private documents streamed only through authenticated routes, SSRF guard on feed URLs, CSV formula neutralisation, timing-safe webhook secret comparison, payment amount and order verified server-side.

6. Remaining work and how to do it

Work through these in order. Items A are required before anyone can pay you. Items B are required for a credible launch. Items C and D can follow in the weeks after launch.

A. Go-live blockers (accounts and credentials)

6.1. Rotate the Anthropic API key and configure production AI

Priority: Critical **Estimated effort:** 30 minutes

Why: The key used during development was shared in a chat and must be treated as compromised. Without a key, AI extraction falls back to the heuristic parser (lower accuracy) and photo extraction is disabled.

What to do:

- 1 Log in to console.anthropic.com, revoke the old key and create a new one restricted to this project.
- 2 Set ANTHROPIC_API_KEY and ANTHROPIC_MODEL in deploy/.env. Keep ANTHROPIC_MODEL=claude-opus-5 unless cost matters more than accuracy, in which case claude-sonnet-5 also passes the tests.
- 3 Set a monthly spend limit in the console. Measured cost is about 0.01 to 0.04 USD per document (5,000 to 8,000 tokens).
- 4 Never commit the key. apps/api/.env and deploy/.env are git-ignored; keep it that way.

Files: deploy/.env, apps/api/src/services/ai.ts

Done when: Uploading a supplier PDF in the supplier portal produces an import with aiUsed=true and per-row confidence scores.

6.2. Server, domain and first deployment

Priority: Critical **Estimated effort:** Half a day

Why: Nothing is reachable by customers until the stack runs on a public domain with HTTPS.

What to do:

- 1 Follow section 4 of this document (and deploy/GO-LIVE.md sections 1 to 3).
- 2 Generate secrets: openssl rand -hex 48 for JWT_SECRET and CARRIER_WEBHOOK_SECRET; a long random POSTGRES_PASSWORD.
- 3 Set DOMAIN, API_DOMAIN and the PLATFORM_* legal values in deploy/.env.
- 4 After the first boot run the xlsx upgrade command listed in GO-LIVE.md section 6b inside the api container image (or bump the version in apps/api/package.json when the registry is reachable) to pick up the patched spreadsheet parser.
- 5 Enable automatic OS security updates and confirm the nightly database backup file appears under deploy/backups.

Files: deploy/docker-compose.prod.yml, deploy/Caddyfile, deploy/scripts/deploy.sh

Done when: https://api.DOMAIN/api/v1/health returns ok and the site loads over HTTPS with a valid certificate.

6.3. Moyasar merchant account (card, Mada and Apple Pay)

Priority: Critical **Estimated effort:** 1 to 2 hours of work, 3 to 10 days waiting for approval

Why: Card payments are fully coded but are disabled until keys exist. Until then checkout offers cash on delivery and bank transfer only.

What to do:

- 1 Apply at moyasar.com with the commercial registration, VAT certificate, IBAN and the website URL. Use test keys immediately while waiting.
- 2 Set MOYASAR_SECRET_KEY, MOYASAR_PUBLISHABLE_KEY and MOYASAR_WEBHOOK_SECRET in deploy/.env and redeploy.

- 3 In the Moyasar dashboard add a webhook pointing to `https://API_DOMAIN/api/v1/payments/webhook/moyasar`, sending the secret in the `x-webhook-secret` header.
- 4 Test with Moyasar test cards: pay an order, confirm the order shows PAID, issue a refund from the admin order page, confirm the webhook marks it REFUNDED.
- 5 Apple Pay: register the domain in Moyasar, download the domain association file and serve it at `/.well-known/apple-developer-merchantid-domain-association` on the web domain (add it to `apps/web/public/.well-known/`).
- 6 Switch to live keys after the approval email; repeat the test with a real card for a small amount and refund it.

Files: `apps/api/src/routes/payments.ts`, `apps/api/src/services/payments.ts`,
`apps/web/src/components/MoyasarForm.tsx`, `apps/web/src/app/pay`

Done when: A live card payment succeeds, the webhook is received (visible in the api logs) and a refund round-trips.

6.4. Unifonic SMS for phone OTP

Priority: High **Estimated effort:** 1 hour plus sender-ID approval (2 to 5 days)

Why: Saudi buyers expect phone sign-in. Without keys, OTP codes are printed to the api log and phone login cannot be used by real users.

What to do:

- 1 Create a Unifonic account, buy SMS credit and request the sender ID MySupplier (requires CR and a letter; CITC approval).
- 2 Set `UNIFONIC_APP_SID` and `UNIFONIC_SENDER_ID` in `deploy/.env` and redeploy.
- 3 Test from the web login page and the mobile app with a real Saudi number. Codes expire in 5 minutes and are rate-limited per phone.

Files: `apps/api/src/services/sms.ts`, `apps/api/src/routes/otp.ts`

Done when: A code arrives on a real phone within a few seconds and login completes.

6.5. Transactional email (SMTP) with SPF and DKIM

Priority: High **Estimated effort:** 1 hour

Why: Order confirmations, bid notifications, invites, password resets and outreach links are sent by email.

What to do:

- 1 Pick a provider (Amazon SES in me-south-1, SendGrid, Mailgun, Zoho or Microsoft 365) and create SMTP credentials.
- 2 Set `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS` and `MAIL_FROM` in `deploy/.env`.
- 3 Add the provider's SPF, DKIM and DMARC DNS records so mail lands in the inbox.
- 4 Trigger a password reset and an RFQ to confirm delivery and that links point to the production domain.

Files: `apps/api/src/services/mailer.ts`, `apps/api/src/services/notifications.ts`

Done when: A test email arrives in Gmail and Outlook without a spam warning.

6.6. Monitoring, uptime and alerts

Priority: High **Estimated effort:** 1 hour

Why: You need to know when the API is down or throwing errors before customers tell you.

What to do:

- 1 Create a Sentry project and set `SENTRY_DSN` in `deploy/.env`. Only the API talks to Sentry; web and mobile post their errors to the API, which forwards them.
- 2 Add an external uptime monitor (Better Uptime, UptimeRobot or Pingdom) on `https://API_DOMAIN/api/v1/health` and the home page, alerting to WhatsApp or email.
- 3 Confirm that the client error endpoint `/api/v1/client-errors` receives web and mobile errors (they appear in Sentry tagged by platform).

Files: apps/api/src/lib/monitoring.ts, apps/api/src/routes/ops.ts

Done when: A forced error appears in Sentry and a simulated outage triggers an alert.

B. Developer work needed for launch

6.7. Carrier integration: contracts, payload mapping and rate cards

Priority: High **Estimated effort:** 1 to 2 days per carrier

Why: Delivery quoting already works from the rate cards, and suppliers can arrange delivery manually. Live booking with a carrier API requires that carrier's contract, credentials and their exact request and response format, which differ by carrier and account.

What to do:

- 1 Sign at least one heavy-goods carrier (TruKKer or Trella for trucks and pallets) and one parcel carrier (SMSA, Aramex or SPL) and obtain API credentials and documentation.
- 2 In apps/api/src/services/carriers/index.ts the generic adapter posts a JSON booking and expects an id and tracking number back. Replace the payload builder and the response mapping in book() with the carrier's real fields, and complete mapStatus() with the carrier's status codes.
- 3 Set the carrier's key in deploy/.env (TRUKKER_API_KEY, TRELLA_API_KEY, SMSA_API_KEY, ARAMEX_API_KEY or SPL_API_KEY). Once set, configured() returns true and shipments book automatically at order confirmation.
- 4 Register the inbound webhook https://API_DOMAIN/api/v1/shipping/webhooks/CARRIER with the carrier, using CARRIER_WEBHOOK_SECRET as the shared secret (header x-webhook-secret).
- 5 Review the seeded rate cards in the admin shipping page against the negotiated contract prices; edit them in the UI or in DEFAULT_RATES for a fresh database.
- 6 Add a test in apps/api/tests/golive.test.ts for the new payload mapping using a recorded carrier response.

Files: apps/api/src/services/carriers/index.ts, apps/api/src/services/shipping.ts, apps/api/src/routes/shipping.ts, apps/web/src/app/admin/shipping

Done when: Confirming a paid order creates a shipment with a real tracking number and status updates arrive through the webhook.

6.8. ZATCA phase 2 (Fatoora) onboarding and XAdES signing

Priority: High for B2B invoices **Estimated effort:** 3 to 5 days plus onboarding time

Why: Phase 1 (QR on invoices) is complete. Phase 2 requires each invoice XML to be signed with a certificate issued by ZATCA and reported or cleared through their API. The XML, hash chain, UUID and counter are implemented; the cryptographic signing and the onboarding are not, because they need the company's real credentials.

What to do:

- 1 Enrol the platform's legal entity in the Fatoora portal, generate a CSR with the EGS unit details and obtain the compliance CSID, then the production CSID.
- 2 Implement XAdES enveloped signing of the UBL XML (ECDSA secp256k1, SHA-256) in apps/api/src/services/einvoice.ts before reportToZatca() sends it. The zatca-xml-js library or the ZATCA SDK (Java) run as a sidecar are both workable; the signing step must also embed the certificate hash in the QR for simplified invoices.
- 3 Set ZATCA_API_BASE (simulation first, then production), ZATCA_BINARY_TOKEN and ZATCA_SECRET in deploy/.env.
- 4 Run the compliance checks in the simulation portal for standard and simplified invoices, credit notes and debit notes, then switch to production.
- 5 Decide whether the platform or each supplier is the invoice issuer for marketplace orders and adjust the seller party in buildUblXml() accordingly (currently the platform issues).

Files: apps/api/src/services/einvoice.ts, apps/api/src/services/zatca.ts, apps/api/src/routes/einvoice.ts

Done when: The e-invoice record status changes from PENDING_CONFIG to REPORTED for a new order and the Fatoora portal shows it.

6.9. Mobile app release to the App Store and Google Play

Priority: High **Estimated effort:** 2 days of work plus review time (Apple 1 to 3 days)

Why: The app is complete and typechecks; it has never been built as a store binary because that needs paid developer accounts.

What to do:

- 1 Enrol in the Apple Developer Program and Google Play Console (both need the company's D-U-N-S or CR).
- 2 Install EAS CLI, run eas login, set the production API base URL in the eas.json profile (EXPO_PUBLIC_API_URL), and run eas build --platform all --profile production.
- 3 Configure push: upload the APNs key to Expo and the FCM credentials for Android (eas credentials). Test push by placing an RFQ bid.
- 4 Add universal links and Android app links for the mysupplier scheme and the production domain (apple-app-site-association and assetlinks.json under apps/web/public/.well-known/).
- 5 Prepare store listings (Arabic and English descriptions, screenshots from the web export, privacy policy URL, data-safety form). Payments go through Moyasar hosted page, which is allowed because goods are physical.
- 6 Submit to TestFlight and the Play internal track first, test on real devices, then release.

Files: apps/mobile/eas.json, apps/mobile/app.json, apps/mobile/src/lib/api.ts

Done when: Both apps are live in the stores and a real order placed from a phone appears in the supplier portal.

6.10. Replace legal placeholders and get the legal pages reviewed

Priority: High **Estimated effort:** 2 hours plus lawyer review

Why: The terms, privacy policy and refund policy are complete drafts, but the company constants contain X placeholders and the texts have not been reviewed by a Saudi lawyer.

What to do:

- 1 Fill the COMPANY object with the real legal name, CR number, VAT number, address, phones and emails.
- 2 Fill PLATFORM_LEGAL_NAME, PLATFORM_VAT_NUMBER, PLATFORM_ADDRESS, BANK_NAME, BANK_IBAN and BANK_BENEFICIARY in deploy/.env; these print on invoices and bank-transfer instructions.
- 3 Have a lawyer review terms, privacy (PDPL compliance), refund policy and the supplier agreement wording on the join page; adjust the text components.
- 4 Register with the Saudi e-commerce authority (Maroof) and show the badge in the footer if desired.

Files: apps/web/src/components/legal/Prose.tsx, apps/web/src/app/terms, apps/web/src/app/privacy, apps/web/src/app/refund-policy, apps/web/src/lib/payments.tsx

Done when: No X placeholder remains in the rendered pages or invoices.

6.11. Object storage for uploads (recommended before scale)

Priority: Medium **Estimated effort:** Half a day

Why: Uploads are stored on the api container's disk volume. That works on one server but is lost if the server is rebuilt without the volume and does not scale to several api instances.

What to do:

- 1 Create an S3-compatible bucket in the Saudi region (AWS me-south-1, Oracle Jeddah, or Cloudflare R2) with public read for product images and private access for company documents.
- 2 In apps/api/src/lib/uploads.ts replace the disk writers behind uploader() and privatePath() with S3 put and signed get calls (aws-sdk v3), keeping the same function signatures so routes do not change.
- 3 Set UPLOAD_BASE_URL to the bucket or CDN public base URL so existing image URLs resolve.
- 4 Migrate the existing uploads folder with aws s3 sync.

Files: apps/api/src/lib/uploads.ts, deploy/docker-compose.prod.yml

Done when: Product images and private documents still load after recreating the api container.

C. Data and content

6.12. Onboard real suppliers and real prices

Priority: Critical for the business **Estimated effort:** Ongoing

Why: The database holds realistic demo suppliers and prices. The platform's value is real, current prices from real suppliers.

What to do:

- 1 Use the admin outreach page: add each supplier company with a contact email or phone; the system emails a magic link where the supplier updates prices without an account.
- 2 For suppliers with price lists, upload their PDF or Excel through the admin imports page; review the AI-extracted rows and publish.
- 3 For suppliers with websites, add a feed (JSON, CSV or an HTML page) in the admin feeds page; it is polled on the schedule you set.
- 4 Invite each supplier to claim their portal account (company invite from the admin company page) so they manage their own inventory and orders.
- 5 Keep OUTREACH_AUTO=true so suppliers with prices older than OUTREACH_STALE_DAYS are asked automatically to refresh.

Files: apps/web/src/app/admin/outreach, apps/web/src/app/admin/imports, apps/web/src/app/admin/feeds

Done when: At least 20 real suppliers with prices updated in the last 14 days across the core categories.

6.13. Curate the materials catalogue

Priority: Medium **Estimated effort:** 1 to 2 days

Why: 270 materials and 32 categories (materials, equipment, machinery, safety, site facilities) cover the common items. Real supplier lists will contain items that need new materials or aliases so that BOQ matching stays accurate.

What to do:

- 1 When import rows are marked unmatched in the review queue, create the material (admin catalogue) and add Arabic and English aliases.
- 2 Extend the synonym tables in apps/api/src/services/boq.ts for local trade names you encounter, and add a test case in apps/api/tests/boq.test.ts for each.
- 3 Add product images for the top 200 materials (the shop shows a category placeholder otherwise).

Files: apps/api/prisma/seed.ts, apps/api/src/services/boq.ts, apps/api/tests/boq.test.ts

6.14. Remove demo data and demo accounts from production

Priority: Critical **Estimated effort:** 30 minutes

Why: Production must not contain the seeded demo accounts with known passwords.

What to do:

- 1 Do not run the seed on production. If it was run on a staging database that becomes production, delete the demo users and companies from the admin console or with a SQL script, and change the admin password immediately.
- 2 Create the real admin user by registering, then promoting the role to ADMIN with one SQL update.

Files: apps/api/prisma/seed.ts, deploy/scripts/seed-demo.sh

D. Hardening and performance (after launch)

6.15. Move shop and BOQ aggregation into SQL when the catalogue grows

Priority: Medium **Estimated effort:** 2 to 3 days

Why: Shop product listing enriches up to 5,000 listings in memory per request, and BOQ matching scans the full catalogue in memory. This is fine for tens of thousands of listings but will slow down

beyond that.

What to do:

- 1 Replace the in-memory enrichment in `services/shop.ts` with a SQL aggregate (min price, offer count, best supplier per material) using a Prisma raw query or a materialised view refreshed every few minutes.
- 2 Add a PostgreSQL full-text index (`pg_trgm` or `tsvector` on `name`, `nameAr` and aliases) and make BOQ candidate selection query the index instead of scanning all materials.
- 3 Cache the public catalogue and best-price endpoints for 60 seconds (in-process LRU or Redis) and set Cache-Control headers for the CDN.

Files: `apps/api/src/services/shop.ts`, `apps/api/src/services/boq.ts`, `apps/api/src/services/pricing.ts`

6.16. Optional security backlog from the audit

Priority: Low **Estimated effort:** 1 to 2 days total

Why: All critical, high and medium findings from the internal audit were fixed. These low-priority items remain and are documented so they are not forgotten.

What to do:

- 1 JWT revocation: add a token version on User and check it in `loadUser`, or move to short access tokens with refresh tokens.
- 2 Two-factor authentication for ADMIN users (TOTP).
- 3 E-invoice counter across several api instances: use a database sequence instead of the in-process counter.
- 4 Device token rebinding: when a push token is registered by a new user, remove it from the previous user.
- 5 Wrap stock deduction at order confirmation in a serialisable transaction to close the small race between concurrent orders.
- 6 Feed URLs: resolve DNS once and connect to the resolved IP to fully prevent DNS-rebinding against `assertPublicUrl`.

Files: `apps/api/src/middleware/auth.ts`, `apps/api/src/services/einvoice.ts`,
`apps/api/src/routes/devices.ts`, `apps/api/src/services/portal.ts`, `apps/api/src/lib/security.ts`

6.17. Real-device QA and accessibility pass

Priority: Medium **Estimated effort:** 2 days

Why: The web app was verified in headless Chromium and the mobile app through its web export and typecheck. Neither has been exercised on physical phones.

What to do:

- 1 Test the mobile app on at least one iPhone and two Android devices (a low-end one included): camera capture for price photos, document picker, push notifications, hosted payment page return, deep links.
- 2 Test the web app on Safari iOS and Chrome Android, including right-to-left rendering of Arabic content and the cookie banner.
- 3 Run Lighthouse on the home, material and shop pages and fix anything below 90 for performance and accessibility.

7. Environment variables reference (deploy/.env)

Variable	Purpose	Where to get it
DOMAIN, API_DOMAIN	Public host names for web and API; Caddy issues certificates for both	Your DNS
POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB	Database credentials used by the db and api containers	Generate
JWT_SECRET, JWT_EXPIRES_IN, APP_SCHEME	Token signing, lifetime (7d), mobile deep-link scheme	openssl rand -hex 48
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, MAIL_FROM	Transactional email	SES, SendGrid, Mailgun, Zoho, Microsoft 365
MOYASAR_SECRET_KEY, MOYASAR_PUBLISHABLE_KEY, MOYASAR_WEBHOOK_SECRET	Card, Mada and Apple Pay payments	moyasar.com dashboard
PLATFORM_LEGAL_NAME, PLATFORM_VAT_NUMBER, PLATFORM_ADDRESS, VAT_RATE	Printed on invoices and ZATCA QR	Company documents
BANK_NAME, BANK_IBAN, BANK_BENEFICIARY	Bank-transfer instructions shown at checkout	Company bank
ANTHROPIC_API_KEY, ANTHROPIC_MODEL	AI extraction of price lists, photos and quotations	console.anthropic.com
OUTREACH_AUTO, OUTREACH_STALE_DAYS	Automatic price-refresh requests to suppliers	Business choice
SENTRY_DSN, SENTRY_TRACES_SAMPLE_RATE	Error and performance monitoring	sentry.io
UNIFONIC_APP_SID, UNIFONIC_SENDER_ID	OTP SMS	unifonic.com
TRUKKER_API_KEY, TRELLA_API_KEY, SMSA_API_KEY, ARAMEX_API_KEY, SPL_API_KEY, CARRIER_WEBHOOK_SECRET	Live carrier booking and inbound status webhooks	Carrier contracts
ZATCA_API_BASE, ZATCA_BINARY_TOKEN, ZATCA_SECRET	Phase 2 e-invoice reporting	Fatoora portal after onboarding
UPLOAD_DIR, UPLOAD_BASE_URL	Upload storage location and public base URL (S3 or CDN)	Server or bucket

8. Suggested three-week plan

Week	Focus	Tasks
Week 1	Infrastructure and accounts	6.1 key rotation, 6.2 server and first deploy, 6.5 email, 6.6 monitoring, 6.14 remove demo data, 6.10 legal placeholders. Start applications for 6.3 Moyasar, 6.4 Unifonic sender ID, 6.9 developer accounts, 6.8 Fatoora.
Week 2	Payments, SMS and suppliers	Finish 6.3 and 6.4 as approvals arrive. Begin 6.12 supplier onboarding with the outreach and import tools. 6.7 first carrier integration. Submit 6.9 builds to TestFlight and Play internal.
Week 3	Launch	6.17 device QA, 6.11 object storage, store release, soft launch with the first suppliers, then public launch. Schedule 6.8 signing and 6.15 performance work for the following sprint.

Rules that protect the platform

- Never commit `deploy/.env` or `apps/api/.env`. Rotate any secret that appears in a chat, ticket or screenshot.
- Keep CI green. Every push runs unit tests, the API smoke script, the web build, the mobile typecheck, the docker build and the browser test.
- Change the database only through committed Prisma migrations.
- Do not seed production. Demo accounts have public passwords.
- Suppliers can never edit shared materials; they only add listings. Keep that rule in `catalogImport.ts` when extending imports.

Where to look when something breaks

Symptom	First place to look
API returns 500	docker compose logs api, then Sentry. Errors include a request id.
Payments not confirming	Moyasar dashboard webhook log; <code>/api/v1/payments/webhook/moyasar</code> must return 200; check <code>MOYASAR_WEBHOOK_SECRET</code> .
OTP codes not arriving	Unifonic balance and sender-ID status; api log prints the code when SMS is not configured.
AI import returns few rows	<code>ANTHROPIC_API_KEY</code> missing or spend limit hit; imports created without AI have <code>aiUsed=false</code> and <code>model</code> null.
Images do not load	<code>UPLOAD_BASE_URL</code> and the uploads volume; helmet cross-origin resource policy is already set to cross-origin.
Emails in spam	SPF, DKIM and DMARC records for the sending domain.